



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

تمرین سوم

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

فهرست مسائل

۱		مسئله ۱
۱		Ubiquitous Language
۱		Domain Service , Application Service
۲		Repository
۳		مسئله ۲
۴		مسئله ۳
۵		مسئله ۴
۶		مسئله ۵
۶		الف
۶		ب
۷		ج
۸		مسئله ۶
۸		الف
۸		ب
۱۰		ج

پاسخ مسئله‌ی ۱.

Ubiquitous Language

زبان مشترک یا Ubiquitous Language یکی از ارکان اصلی و حیاتی در رویکرد DDD است. این مفهوم به یک واژه‌نامه و زبان دقیق اشاره دارد که توسط تمام اعضای تیم، شامل متخصصان کسب‌وکار (Domain Experts) و توسعه‌دهندگان نرم‌افزار، به صورت توافقی ساخته و استفاده می‌شود. هدف اصلی این است که شکاف ارتباطی و سوءتفاهم‌های ناشی از ترجمه زبان تجاری به زبان فنی از بین برود. در این رویکرد، اصطلاحاتی که در جلسات و مکالمات روزمره تیم به کار می‌روند، باید دقیقاً همان نام‌هایی باشند که در طراحی مدل‌ها، کلاس‌ها، متدها و حتی متغیرها در سورس‌کد استفاده می‌شوند. این همگام‌سازی تضمین می‌کند که نرم‌افزار دقیقاً بازتاب‌دهنده نیازها و مفاهیم واقعی کسب‌وکار باشد.

Domain Service , Application Service

در معماری DDD برای جلوگیری از در هم آمیختگی وظایف، سرویس‌ها به دو لایه مجزا با اهداف متفاوت تقسیم می‌شوند.

سرویس دامنه (Domain Service) دربرگیرنده آن دسته از قوانین و منطق‌های تجاری (Business Logic) است که به طور ذاتی به یک موجودیت (Entity) یا شیء مقداری (Value Object) خاص تعلق ندارند. این سرویس‌ها معمولاً عملیاتی را مدل می‌کنند که نیازمند تعامل بین چندین شیء مختلف در دامنه است. سرویس‌های دامنه کاملاً درگیر هسته کسب‌وکار هستند و هیچ‌گونه حالت (State) پایداری را در خود نگه نمی‌دارند.

در مقابل، سرویس کاربردی (Application Service) فاقد هرگونه منطق تجاری است و صرفاً نقش یک هماهنگ‌کننده (Orchestrator) را ایفا می‌کند. وظیفه اصلی این سرویس، دریافت درخواست‌ها از لایه‌های بیرونی نرم‌افزار، مدیریت تراکنش‌ها، بررسی دسترسی‌ها و سپس فراخوانی اشیاء و سرویس‌های لایه دامنه برای انجام کار اصلی است. به زبان ساده، Use Cases سیستم را پیاده‌سازی می‌کند تا دامنه بتواند بدون درگیری با مفاهیم زیرساختی، فقط روی منطق خالص خود تمرکز کند.

Repository

الگوی مخزن یا Repository ابزاری برای جداسازی منطق خالص دامنه از جزئیات فنی مربوط به ذخیره‌سازی و بازیابی داده‌ها (مانند دیتابیس‌ها) است. در دیدگاه DDD، یک مخزن شبیه به یک مجموعه داده درون‌حافظه‌ای (In-Memory Collection) عمل می‌کند که صرفاً برای ذخیره و بازیابی ریشه‌های تجمیع (Aggregate Roots) طراحی شده است. مدل دامنه نباید هیچ‌گاه درگیر نحوه نوشتن کوئری‌های SQL یا ارتباط با ORM‌ها شود؛ بلکه تنها از طریق رابط‌ها (Interfaces) با مخزن صحبت می‌کند. مخزن در پس‌زمینه، پیچیدگی‌های ارتباط با زیرساخت داده را پنهان می‌کند و به توسعه‌دهندگان اجازه می‌دهد تا هسته نرم‌افزار را کاملاً مستقل و ایزوله از تکنولوژی‌های دیتابیس نگه دارند.

پاسخ مسئله‌ی ۲.

تفاوت اصلی و بنیادین بین موجودیت (Entity) و شیء مقداری (Value Object) در مفهوم «هویت» (Identity) نهفته است. یک Entity دارای یک شناسه یکتا است که در طول زمان و کاملاً مستقل از تغییر ویژگی‌هایش، هویت آن را ثابت نگه می‌دارد. به این معنا که اگر تمام مشخصات دو Entity یکسان باشد اما شناسه‌های متفاوتی داشته باشند، کاملاً دو شیء مجزا محسوب می‌شوند (مثل دو انسان با مشخصات ظاهری کاملاً یکسان اما کدملی متفاوت). در مقابل، یک Value Object فاقد هرگونه هویت مستقل است و تماماً با ویژگی‌ها و مقادیرش تعریف می‌شود. اگر دو Value Object دارای مقادیر برابری باشند، از نظر سیستم کاملاً یکسان و قابل جایگزینی با یکدیگر هستند (مانند دو اسکناس ده هزار تومانی که برای شما تفاوتی ندارند کدام را خرج کنید). همچنین، Value Object ها بر خلاف Entity ها باید به صورت تغییرناپذیر (Immutable) طراحی شوند.

در خصوص شرایط استفاده، شما زمانی باید از یک Entity استفاده کنید که نیاز به پیگیری چرخه حیات (Lifecycle) و تغییرات یک شیء در طول زمان دارید و تمایز آن شیء از سایر اشیاء مشابه برای منطق کسب و کار شما حیاتی است (مانند یک سفارش خرید یا حساب کاربری). اما زمانی باید از Value Object استفاده کنید که هدف شما صرفاً توصیف، اندازه‌گیری یا کمی‌سازی یک ویژگی در دامنه است و برایتان مهم نیست که دقیقاً کدام نمونه از آن شیء را در دست دارید (مانند آدرس پستی، مبلغ پول یا یک بازه زمانی). به عنوان یک قانون کلی در رویکرد DDD، توصیه می‌شود تا جای ممکن مفاهیم را به جای Entity به صورت Value Object مدل‌سازی کنید؛ زیرا به دلیل تغییرناپذیری، ایمن‌تر هستند، عوارض جانبی (Side-effects) ندارند و مدیریت پیچیدگی سیستم را بسیار آسان‌تر می‌کنند.

پاسخ مسئله‌ی ۳.

خیر، طراحی یک مدل یکپارچه برای مفهوم Customer در کل سیستم یک اشتباه معماری است و در DDD یک Anti-pattern محسوب می‌شود. تلاش برای گنجاندن تمام ویژگی‌های مربوط به فروش، حسابداری، انبار و پشتیبانی در یک موجودیت واحد، منجر به ایجاد یک God Class می‌شود. این رویکرد باعث درهم‌تنیدگی شدید تیم‌ها و کدها شده و نگهداری سیستم را به شدت دشوار می‌کند؛ زیرا هر تغییری از سوی یک دپارتمان می‌تواند بر کار سایر دپارتمان‌ها تأثیر منفی بگذارد و توافق بر سر یک زبان مشترک را عملاً غیرممکن می‌سازد.

بر اساس مفهوم Bounded Context (زمینه محدود)، سیستم باید به مرزهای معنایی کاملاً مشخص و ایزوله تقسیم شود که در اینجا هر دپارتمان (Sales، Accounting و Support) یک Bounded Context مجزا به شمار می‌رود. در داخل هر یک از این مرزها، موجودیت Customer معنای خاص خود را دارد و تنها شامل ویژگی‌هایی است که برای آن دپارتمان اهمیت دارد. به این ترتیب، ما در سطح کد چندین کلاس مجزا به نام Customer (یا نام‌های دقیق‌تر مانند SalesCustomer و AccountingCustomer) خواهیم داشت که هر کدام با Ubiquitous Language کانتکست خودشان طراحی شده‌اند و تیم‌های توسعه می‌توانند آن‌ها را کاملاً مستقل از یکدیگر تکامل دهند.

برای یکپارچه‌سازی و ارتباط بین این مدل‌های متفاوت در کانتکست‌های مختلف، باید از الگوهای Context Mapping استفاده کرد. رایج‌ترین و بهترین رویکرد، اشتراک‌گذاری یک شناسه یکتای جهانی (Global Customer ID) بین تمامی سیستم‌هاست. ارتباطات نیز غالباً از طریق معماری Event-Driven پیاده‌سازی می‌شود؛ به این صورت که وقتی مشتری جدیدی ثبت نام می‌کند، یک Domain Event مانند CustomerRegistered منتشر می‌شود و سایر کانتکست‌ها (مثل حسابداری یا پشتیبانی) با دریافت این رویداد، مدل اختصاصی خودشان از آن مشتری را با همان شناسه مشترک در دیتابیس خود ایجاد می‌کنند. همچنین در مواقعی که یک کانتکست نیاز به خواندن مستقیم داده از کانتکست دیگری با مدلی متفاوت دارد، از الگوی Anti-Corruption Layer استفاده می‌شود تا داده‌های دریافتی، پیش از ورود به مرز کانتکست مقصد، به مدل بومی آن ترجمه شوند.

پاسخ مسئله‌ی ۴.

بروز خطا در این سناریو ناشی از پدیده Mental Translation و نبود یک درک یکپارچه از مفاهیم است. وقتی تیم فنی اصطلاح کسب‌وکار (Customer) را به دلخواه خود به اصطلاح دیگری (Client) در کد نگاشت می‌کند، تفاوت‌های ظریف و حیاتی دامنه از بین می‌رود. از آنجا که طبق منطق کسب‌وکار یک Customer (شخصیت حقوقی یا پرداخت‌کننده) می‌تواند شامل چندین Client (کاربر نهایی) باشد، اگر توسعه‌دهندگان این دو مفهوم مجزا را در کد قالب یک شیء واحد در نظر بگیرند، قوانین تجاری دچار فروپاشی می‌شوند. به عنوان مثال، فرآیندهایی مانند صدور فاکتور یا بررسی محدودیت‌های مالی که باید در سطح Customer اعمال شوند، به اشتباه در سطح تک‌تک Client‌ها ارزیابی شده و منجر به باگ‌های منطقی و محاسباتی مخرب در سیستم می‌شوند.

مفهوم Ubiquitous Language این مشکل را با اجبار کردن تمامی اعضا به استفاده از یک واژگان واحد، دقیق و توافق‌شده حل می‌کند. برای برطرف کردن این سوءتفاهم در سطح تیم، Domain Experts و توسعه‌دهندگان باید جلسات مشترکی برگزار کنند تا یک واژه‌نامه صریح بسازند و هرگونه ابهام را برطرف کنند. قانون اصلی این است که ترجمه کردن کلمات بین تیم‌ها باید کاملاً متوقف شود؛ اگر کسب‌وکار بین پرداخت‌کننده و کاربر نهایی تفاوت قائل است، تیم فنی نیز ملزم است دقیقاً از همان کلمات Customer و Client با همان تعاریف تجاری در تمامی مکالمات روزمره، مستندات و جلسات طراحی استفاده کند.

در سطح مدل دامنه نیز این توافق زبانی باید مستقیماً و بی‌واسطه در سورس‌کد پیاده‌سازی شود (Model-Driven Design). تیم توسعه باید کدها را Refactor کرده و دو مفهوم کاملاً مجزا در مدل ایجاد کند. کلاس Customer باید مدل‌سازی شود تا صرفاً دربرگیرنده منطق مالی، قراردادهای پرداخت‌ها باشد و در کنار آن کلاس Client برای مدیریت ویژگی‌های مربوط به دسترسی و استفاده از سیستم طراحی گردد. همچنین ارتباط ساختاری بین این دو (مثلاً اینکه یک Customer به عنوان یک ریشه تجمیع، مدیریت لیستی از Client‌های زیرمجموعه خود را بر عهده دارد) باید به روشنی در کد پیاده شود تا نرم‌افزار به آینه‌ای تمام‌نما از واقعیت کسب‌وکار تبدیل گردد.

پاسخ مسئله‌ی ۵.

الف

ابزارهای خودکار شناسایی میکروسرویس (مانند Mono2Micro) معمولاً معماری‌های کاندیدا را صرفاً بر اساس تحلیل توالی دسترسی به داده‌ها، وابستگی‌های کدی و لاگ‌های اجرایی تولید می‌کنند. دلیل ناکافی بودن آن‌ها برای DDD این است که خروجی این ابزارها فاقد درک معنایی از منطق کسب و کار است و خروجی آن‌ها قابلیت مدل‌سازی یا بازآرایی با مفاهیم غنی DDD را ندارد. به عبارت دیگر، این ابزارها یک برش فنی از سیستم ارائه می‌دهند اما شکاف بزرگی بین این ساختارهای استخراج‌شده و الگوهای معنایی دامنه وجود دارد که معماران برای توسعه سیستم به آن نیاز دارند.

ب

در ابزار Mono2Micro، معیار اصلی برای کنار هم قرار دادن موجودیت‌ها و تشکیل یک Cluster، مفهوم Transac-tional Similarity است؛ به این معنا که موجودیت‌هایی که معمولاً در جریان یک تراکنش به صورت همزمان خوانده یا نوشته می‌شوند، با احتمال بیشتری در یک خوشه قرار می‌گیرند.

هنگامی که محققان تلاش کردند این خوشه‌ها را به مفاهیم DDD نگاشت کنند، مشخص شد که هر خوشه به طور همزمان با دو مفهوم متفاوت در DDD همپوشانی معنایی دارد. از یک سو، این خوشه‌ها از نظر رفتاری به Aggregate شباهت دارند، زیرا در DDD نیز Aggregate مجموعه‌ای از اشیاء دامنه است که به شدت به هم وابسته‌اند و برای اعمال تغییرات، به صورت یک واحد تراکنشی یکپارچه عمل می‌کنند. از سوی دیگر، همین خوشه‌ها از منظر ساختاری به Bounded Context نیز نزدیک هستند، زیرا مرزهای تعامل و وابستگی بین بخش‌های مختلف سیستم را مشخص می‌کنند و عملاً نقش جداسازی در سطح معماری و میکروسرویس را ایفا می‌نمایند.

با توجه به این ماهیت دوگانه، راهکار نهایی ارائه‌شده در مقاله این است که هر خوشه استخراج‌شده از Mono2Micro به یک Bounded Context مجزا تبدیل شود و درون هر Bounded Context تنها یک Aggregate منفرد و نسبتاً بزرگ تعریف گردد که تمامی موجودیت‌های آن خوشه را در بر بگیرد. این تصمیم به عنوان یک نقطه شروع برای مدل‌سازی در نظر گرفته می‌شود و نه یک طراحی نهایی ثابت، زیرا به معمار اجازه می‌دهد ابتدا ساختار کلی سیستم را بر اساس داده‌های رفتاری واقعی شکل دهد و سپس آن را به صورت تدریجی بهینه‌سازی کند.

مقاله همچنین توضیح می‌دهد که سایر ترکیب‌های ممکن چرا کنار گذاشته شده‌اند؛ اگر هر خوشه صرفاً به یک Aggregate تبدیل می‌شد و همه در یک Bounded Context بزرگ قرار می‌گرفتند، ماهیت توزیع‌شده و استقلال سرویس‌ها که هدف اصلی معماری میکروسرویس است از بین می‌رفت. از طرف دیگر، اگر هر خوشه فقط به یک Bounded Context تبدیل می‌شد اما درون آن چندین Aggregate مستقل تعریف می‌گردید، ارتباطات تراکنشی واقعی که باعث شکل‌گیری آن خوشه شده‌اند در مدل نهایی بازتاب دقیقی نداشتند و انسجام رفتاری سیستم از دست می‌رفت.

در ادامه، مقاله تأکید می‌کند که این نگاشت اولیه قابل تکامل است. معمار می‌تواند بر اساس دانش دامنه و تحلیل دقیق‌تر، Aggregate اولیه را بازآرایی و به چند Aggregate کوچک‌تر و معنایی‌تر تقسیم کند. همچنین برای مدیریت وابستگی‌ها پس از تفکیک کانتکست‌ها، به جای ارجاعات مستقیم بین موجودیت‌ها، از موجودیت‌های مرجع محلی استفاده می‌شود تا مرزهای Bounded Context حفظ شود و در عین حال امکان شناسایی و بازطراحی ارتباطات بین بخش‌های مختلف سیستم برای تیم معماری فراهم بماند.

ج

ابزار Context Mapper در این پایپ‌لاین نقش موتور مدل‌سازی و تولید زبان را بر عهده دارد. این ابزار خروجی‌های Mono2Micro را دریافت کرده و با استفاده از زبان دامنه اختصاصی خود به نام CML، آن‌ها را به صورت خودکار به مدل‌های DDD ترجمه می‌کند. مزیت کلیدی این رویکرد برای معمار نرم‌افزار این است که با در اختیار داشتن یک مدل واضح، متنی و بصری (CML)، می‌تواند پیچیدگی‌های سیستم استخراج‌شده را به راحتی درک کند و مرزهای کانتکست‌ها را با استفاده از الگوهای استراتژیک و تاکتیکی DDD به شکلی چابک‌تر ویرایش، بازآرایی و بهینه‌سازی نماید.

پاسخ مسئله‌ی ۶.

الف

مفهوم CQRS: الگوی CQRS (سرنام Command Query Responsibility Segregation) به معنای جداسازی مسئولیت فرمان‌ها (Commands) که حالت سیستم را تغییر می‌دهند، از پرس و جوها (Queries) که تنها داده می‌خوانند، است:

- **Command:** نیت تغییر حالت را بیان می‌کند (مانند «این لینک را بساز» یا «این لینک را غیرفعال کن»)
فرمان‌ها هیچ داده‌ای جز نتیجه‌ی موفقیت یا شکست بر نمی‌گردانند.
- **Query:** تنها داده می‌خواند و هیچ‌گاه حالت دامنه را تغییر نمی‌دهد (بدون اثر جانبی).

در این پیاده‌سازی، این تفکیک حتی در ساختار کد نیز آشکار است: مسیر نوشتن در بسته‌ی application/command (هندلرهای CreateLink، DisableLink و RecordClick) و مسیر خواندن در بسته‌ی application/query (هندلرهای Redirect و GetStats) قرار دارد.

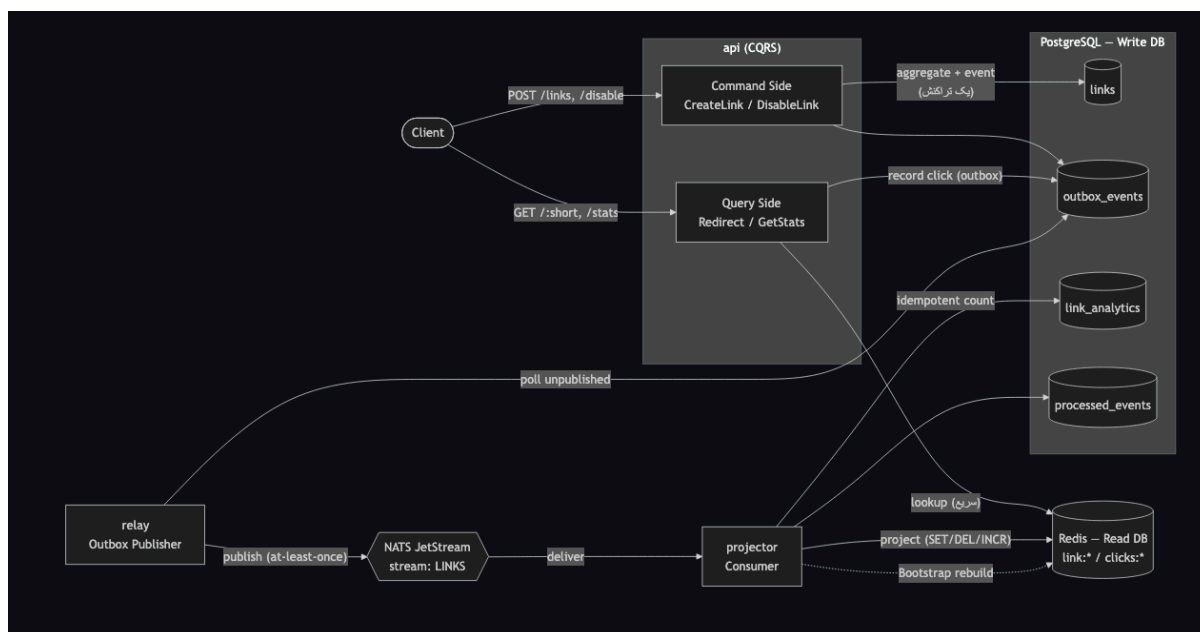
دلیل استفاده از Write DB و Read DB مجزا: هسته‌ی فلسفه‌ی CQRS این است که نیازهای سمت نوشتن و سمت خواندن ذاتا متفاوت‌اند و یک مدل واحد نمی‌تواند هر دو را به‌خوبی برآورده کند. مهم‌ترین دلایل عبارت‌اند از:

- **بهینه‌سازی مستقل مدل‌ها:** مدل نوشتن باید قواعد دامنه و یکپارچگی تراکنشی را رعایت کند؛ به‌همین دلیل از PostgreSQL رابطه‌ای استفاده شده است. مدل خواندن تنها باید سریع باشد؛ Redis با دسترسی $O(1)$ برای هدایت و شمارش کلیک ایده‌آل است.
- **مقیاس‌پذیری ناهم‌سان:** در یک کوتاه‌کننده‌ی لینک، تعداد هدایت‌ها (خواندن) چندین مرتبه بزرگ‌تر از ساخت لینک (نوشتن) است. با دو دیتابیس می‌توان هر سمت را جداگانه مقیاس داد.
- **جداسازی بار (کاهش contention):** بار سنگین خواندن، تراکنش‌های نوشتن را کند نمی‌کند و برعکس.
- **شکل‌دهی داده برای مصرف:** داده در Read DB دقیقاً به همان شکلی ذخیره می‌شود که پاسخ آماده باشد (نگاشت مستقیم shortCode به longURL و شمارنده‌ی کلیک)؛ پس در زمان خواندن نیازی به join یا محاسبه نیست.

از آنجا که نوشتن در Write DB و به‌روزرسانی Read DB دو عملیات جدا هستند، Model Reader با تأخیری کوتاه به‌روز می‌شود؛ یعنی Eventual Consistency. این یک trade-off آگاهانه است: در ازای کارایی و مقیاس‌پذیری بالا، می‌پذیریم که آمار کلیک یا انتشار یک لینک تازه‌ساخته‌شده با تأخیری کوتاه در سمت خواندن نمایان شود.

ب

شکل ۱ نمودار مؤلفه‌های سامانه را نشان می‌دهد. سیستم از سه فرایند مجزا (که از یک image واحد ساخته می‌شوند) و سه زیرساخت داده‌ای تشکیل شده است.

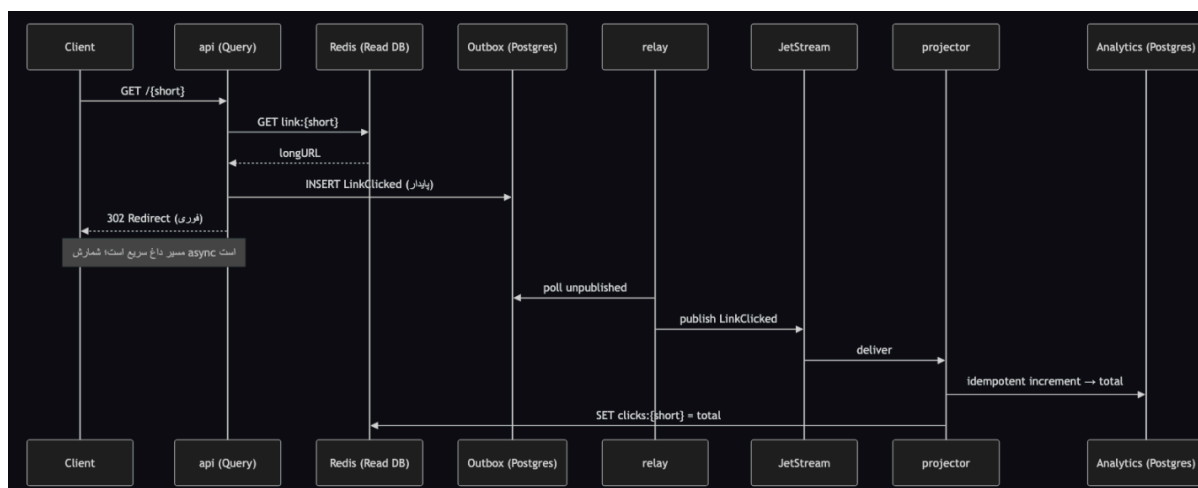


شکل ۱: نمودار مؤلفه‌های سامانه: تفکیک سمت نوشتن (Command) و خواندن (Query)، الگوی Outbox، صف JetStream و Projector.

نقش هر مؤلفه به‌صورت زیر است:

- **api:** هم سمت Command (درخواست‌های POST) و هم سمت Query (درخواست‌های GET) را سرو می‌کند، اما در کد به دو لایه‌ی کاملاً جدا تقسیم شده است.
- **PostgreSQL (Write DB):** منبع حقیقت؛ شامل وضعیت aggregate‌ها، جدول outbox، شمارنده‌ی پایدار کلیک و جدول حذف تکراری.
- **relay:** ردیف‌های منتشرنشده‌ی جدول outbox را به‌ترتیب می‌خواند و روی JetStream منتشر می‌کند (نیمه‌ی انتشار الگوی Transactional Outbox).
- **NATS JetStream:** گذرگاه پیام بادوام (با ذخیره‌سازی روی دیسک) که تحویل at-least-once و حذف تکراری سمت سرور را فراهم می‌کند.
- **projector:** مصرف‌کننده‌ی بادوام و idempotent رویدادها؛ شمارنده‌ی پایدار را در PostgreSQL و Model Reader سریع را در Redis به‌روز می‌کند و هنگام راه‌اندازی، Model Reader را از روی منبع حقیقت بازسازی می‌کند.
- **Redis (Read DB):** Model Reader سریع شامل نگاشت link:{short} به longURL و شمارنده‌ی clicks:{short}.

جریان یک عملیات کامل (هدایت و ثبت کلیک): شکل ۲ نمودار توالی مسیر هدایت را نشان می‌دهد. هنگام مراجعه به یک لینک کوتاه، api ابتدا مقصد را با یک خواندن سریع از Redis می‌یابد و بلافاصله پاسخ 302 را برمی‌گرداند؛ هم‌زمان یک رویداد LinkClicked را به‌صورت پایدار در جدول outbox درج می‌کند. سپس relay این رویداد را به JetStream منتشر می‌کند و projector آن را مصرف کرده، شمارنده‌ی پایدار را افزایش می‌دهد و مقدار به‌روزشده را در Redis می‌نویسد. به این ترتیب مسیر داغ هدایت سریع می‌ماند و شمارش کلیک به‌صورت ناهم‌گام و بدون گم‌شدن انجام می‌شود.



شکل ۲: نمودار توالی مسیر هدایت و ثبت کلیک به صورت ناهم‌گام (Eventual Consistency).

ج

کانتینرهای سامانه:

نقش	فناوری	سرویس
رابط REST؛ سمت فرمان و پرس‌وجو	Go + Gin	api
انتشار رویدادها از outbox به JetStream	Go	relay
مصرف رویدادها و به‌روزرسانی و بازسازی Model Reader	Go	projector
Write DB (منبع حقیقت)	PostgreSQL 15	postgres
Read DB (Model Reader سریع)	Redis 7 (AOF)	redis
گذرگاه پیام بادوام	NATS JetStream	nats

جدول ۱: کانتینرهای سامانه و نقش هر یک.

مدل داده: جدول‌های سمت نوشتن در PostgreSQL و کلیدهای سمت خواندن در Redis در جدول ۲ آمده‌اند.

نقش	جدول / کلید	دیتابیس
وضعیت aggregate (کد، URL، وضعیت، نسخه)	links	Postgres
صف بادوام رویدادها (الگوی Outbox)	outbox_events	Postgres
شمارنده‌ی پایدار کلیک (منبع حقیقت آمار)	link_analytics	Postgres
حذف تکراری برای idempotency	processed_events	Postgres
نگاشت کد کوتاه به URL اصلی	link:{short}	Redis
شمارنده‌ی کلیک برای پاسخ سریع	clicks:{short}	Redis

جدول ۲: مدل داده‌ی سمت نوشتن و سمت خواندن.

الگوی Transactional Outbox و مسئله‌ی Dual-Write: اگر هنگام ساخت لینک، ابتدا در دیتابیس بنویسیم و سپس رویداد را روی صف منتشر کنیم، دو نوشتن مستقل داریم و اگر انتشار به‌خاطر کرش یا قطعی شبکه ناموفق

شود، رویداد برای همیشه گم می‌شود و دو دیتابیس واگرا می‌گردند. راه‌حل این است که رویداد در همان تراکنش دیتابیزی تغییر aggregate در جدول outbox نوشته شود؛ پس یا هر دو ثبت می‌شوند یا هیچ‌کدام:

```

1 // persist aggregate state AND its events in a single transaction
2 return r.db.Transaction(func(tx *gorm.DB) error {
3     if err := tx.Save(&model).Error; err != nil {           // aggregate state
4         return err
5     }
6     for _, e := range events {                               // events in same TX
7         if err := tx.Create(&outboxRow(e)).Error; err != nil {
8             return err
9         }
10    }
11    return nil
12 })

```

سپس فرایند relay ردیف‌های منتشرنشده را خوانده و پس از انتشار موفق روی JetStream، آن‌ها را published علامت می‌زند. اگر relay وسط کار کرش کند، چون ردیف هنوز علامت نخورده، در دور بعد دوباره منتشر می‌شود (تحویل at-least-once).

Idempotency: چون تحویل at-least-once است، ممکن است یک رویداد بیش از یک‌بار به projector برسد. برای جلوگیری از شمارش دوباره، شناسه‌ی هر رویداد در جدول processed_events ثبت می‌شود و تنها در صورتی که رویداد تازه باشد، شمارنده افزایش می‌یابد؛ این کار به‌صورت یک تراکنش اتمی انجام می‌گیرد.

پایداری و بازسازی‌پذیری: منبع حقیقت برای نگاشت لینک‌ها و شمارنده‌ی کلیک، PostgreSQL با volume پایدار است و Redis تنها یک projection به‌شمار می‌رود. اگر کل Redis پاک شود، projector هنگام راه‌اندازی با اجرای تابع Model Reader Bootstrap را به‌طور کامل از روی جدول‌های links و link_analytics باز می‌سازد. این رفتار به‌صورت عملی آزموده شده است: پس از اجرای FLUSHALL روی Redis و ری‌استارت projector، هم نگاشت لینک‌ها و هم شمارنده‌ی کلیک‌ها دقیقاً از داده‌های پایدار PostgreSQL بازیابی شدند.

مفاهیم DDD به‌کاررفته: کل سامانه یک Bounded Context است و الگوهای تاکتیکی زیر در آن پیاده شده‌اند:

aggregate غنی Link با فیلدهای خصوصی و invariant «لینک غیرفعال را نمی‌توان دوباره غیرفعال کرد»؛ Value Object‌های خوداعتبارسنج (LinkID، ShortCode، URL و LinkStatus)؛ رویدادهای دامنه (LinkCreated، LinkDisabled و LinkClicked)؛ و الگوی Ports & Adapters که در آن دامنه هیچ وابستگی‌ای به زیرساخت ندارد (Persistence و Ignorance).